

凸包

求两个凸多边形之间的最大/最小距离： $C = A - B$ 距离原点最远/最近的点

```
1 struct Convex: Polygon { // 凸多边形
2     // 闵可夫斯基和, O(N + M)
3     Convex operator+(const Convex &c) const {
4         const auto &p = this->p;
5         vector<Segment> e1(p.size()), e2(c.p.size()), edge(p.size() +
6 c.p.size());
7         vector<Point> res; res.reserve(p.size() + c.p.size());
8         const auto cmp = [] (const Segment &u, const Segment &v) {
9             return Argcmp() (u.b - u.a, v.b - v.a);
10        };
11        for(size_t i = 0; i < p.size(); i++) e1[i] = {p[i], p[this->nxt(i)]};
12        for(size_t i = 0; i < c.p.size(); i++) e2[i] = {c.p[i],
13 c.p[c.nxt(i)]};
14        rotate(e1.begin(), min_element(e1.begin(), e1.end(), cmp), e1.end());
15        rotate(e2.begin(), min_element(e2.begin(), e2.end(), cmp), e2.end());
16        merge(e1.begin(), e1.end(), e2.begin(), e2.end(), edge.begin(), cmp);
17        const auto check=[] (const vector<Point> &res, const Point &u) {
18            const auto back1 = res.back(), back2 = *prev(res.end(), 2);
19            return (back1-back2).toleft(u-back1) == 0 && (back1-back2) * (u-
20 back1) >= -eps;
21        };
22        auto u=e1[0].a + e2[0].a;
23        for(const auto &v: edge) { // 去除共线点
24            while(res.size() > 1 && check(res,u)) res.pop_back();
25            res.push_back(u);
26            u = u + v.b - v.a;
27        }
28        if(res.size() > 1 && check(res, res[0])) res.pop_back();
29        return {res};
30    }
31
32    void convexhull() { // Andrew 求凸包, O(n logn)
33        auto &p = this->p;
34        sort(p.begin(), p.end());
35        p.erase(unique(p.begin(), p.end()), p.end());
36        if(p.size() <= 2) return;
37        vector<Point> stk;
38        const auto check=[] (const vector<Point> &stk, const Point &u) {
39            const auto back1 = stk.back(), back2 = *prev(stk.end(), 2);
40            return (back1 - back2).toleft(u - back1) <= 0;
41        };
42    }
```

```

39     for(const Point &u: p) {
40         while(stk.size() > 1 && check(stk, u))
41             stk.pop_back();
42         stk.push_back(u);
43     }
44     size_t sz = stk.size();
45     p.pop_back(); reverse(p.begin(), p.end());
46     for(const Point &u:p) {
47         while(stk.size() > sz && check(stk, u))
48             stk.pop_back();
49         stk.push_back(u);
50     }
51     stk.pop_back();
52     p = move(stk);
53 }
54
55 T rotcaliper() const { // 旋转卡壳求凸包直径平方, O(n)
56     const auto &p = this->p;
57     if(p.size() == 1) return 0;
58     if (p.size() == 2) return p[0].dis2(p[1]);
59     const auto area = [](const Point &u, const Point &v, const Point &w) {
60         return (w - u) ^ (w - v);
61     };
62     T ans = 0;
63     for(size_t i = 0, j = 1, sz = p.size(); i < sz; i++) {
64         const auto nxti = this->nxt(i);
65         ans = max({ans, p[j].dis2(p[i]), p[j].dis2(p[nxti])});
66         while(area(p[this->nxt(j)], p[i], p[nxti]) >= area(p[j], p[i],
p[nxti])) {
67             j = this->nxt(j);
68             ans = max({ans, p[j].dis2(p[i]), p[j].dis2(p[nxti])});
69         }
70     }
71     return ans;
72 }
73
74 // 极角序上二分判断点 a 是否在凸包内, O(log n)
75 // -1: 在凸包边上; 0: 在凸包外; 1: 在凸包内
76 int is_in(const Point &a) const {
77     const auto &p = this->p;
78     if(p.size() == 1) return a == p[0] ? -1: 0;
79     if(p.size() == 2) return Segment{p[0], p[1]}.is_on(a) ? -1: 0;
80     if(a == p[0]) return -1;
81     if((p[1]-p[0]).toleft(a-p[0])==-1 || (p.back()-p[0]).toleft(a-
p[0])==1) return 0;
82     const auto cmp = [&](const Point &u, const Point &v) {
83         return (u - p[0]).toleft(v - p[0]) == 1;

```

```

84         };
85         // lower_bound(first, last, value, cmp) 返回第一个 cmp(p[pos], value) 为
false
86         const size_t i = lower_bound(p.begin() + 1, p.end(), a, cmp) -
p.begin();
87         if(i == 1) return Segment{p[0], p[i]}.is_on(a) ? -1: 0;
88         if(i == p.size() - 1 && Segment{p[0], p[i]}.is_on(a)) return -1;
89         if(Segment{p[i - 1], p[i]}.is_on(a)) return -1;
90         return (p[i] - p[i - 1]).toleft(a - p[i - 1]) > 0;
91     }
92
93     // 寻找凸多边形在给定方向函数上的极值点, O(log n)
94     template<typename F> size_t extreme(const F &dir) const {
95         const auto &p = this->p;
96         const auto check = [&](const size_t i) {
97             return dir(p[i]).toleft(p[this->nxt(i)] - p[i]) >= 0;
98         };
99         const auto dir0 = dir(p[0]); const auto check0 = check(0);
100        if(!check0 && check(p.size() - 1)) return 0;
101        const auto cmp = [&](const Point &v) {
102            const size_t vi = &v - p.data();
103            if(vi == 0) return 1;
104            const auto checkv = check(vi);
105            const auto t = dir0.toleft(v - p[0]);
106            if(vi == 1 && checkv == check0 && t == 0) return 1;
107            return checkv ^ (checkv == check0 && t <= 0);
108        };
109        return partition_point(p.begin(), p.end(), cmp) - p.begin();
110    }
111    // 过凸包外一点 a, 求到凸多边形的两条切线切点, 返回切点索引
112    pair<size_t, size_t> tangent(const Point &a) const {
113        const size_t i = extreme([&](const Point &u){return u-a;});
114        const size_t j = extreme([&](const Point &u){return a-u;});
115        return {i, j};
116    }
117    // 求平行于给定直线 a 的两条凸包切线切点, 返回切点索引
118    pair<size_t, size_t> tangent(const Line &a) const {
119        const size_t i = extreme([&](...){return a.v;});
120        const size_t j = extreme([&](...){return -a.v;});
121        return {i, j};
122    }
123 };

```

